状態保持型エージェントの API 説明省略における仕事全体の費用と終了手続きの診断

検討用原稿・著者情報未設定

2026 年 10 月 23 日

あらまし　状態保持型エージェントの 4 課題 12 条件で API 説明の履歴省略を比較した。詳細省略の既知費用は全表示を上回った。写真整理では同一作業状態でも完了通知で成否が変わった。仕事全体の費用と終了手続きを分けて報告する。

検討範囲. 既知課題の実証原稿であり，新手法の有効性を主張するものではない．中心的貢献，著者情報，公式様式への適合は未確定である．

1　目的と検証する問い

AI エージェントによる仕事の費用を減らすには，必要な情報・処理と最終品質を保ち，取得，生成，実行，検証を含む全体を制御する必要がある．入力を短くしても，後続の再取得や生成が増えれば費用は減らない [1, 2]．また，成功判定の差だけでは，実作業が改善したか，終了手続きが変わったかを区別できない．本稿は，情報表示の省略を評価する際のこの二つの問題を，状態保持型の実行環境で調べる．

API を使う作業では，何ができるかを示す一覧と，引数を説明する詳細の両方が現れる．一度取得した説明を履歴へ繰り返す負担を減らすため，本稿では両者の表示を別々に省く．ただし，Python 状態，公開スキーマと原文への参照経路を保持するため，表示の省略は実行環境からの情報消去とは異なる．

問いは，**共通の途中状態から API 説明の表示を省いたとき，仕事全体の費用と最終成否がどう変わり，その成否差を実作業の改善と解釈できるか**である．既知 4 課題の全 12 条件を比較し，その後，写真整理の保存状態に完了通知だけを加える診断を行う．入力短縮，課題の成功数，作業状態を別々に記録することで，今回の観測から支持できる費用・品質の解釈を限定する．

2　関連研究と本稿の範囲

Beyond Token Savings[1] は，複数モデルと反復を用いて圧縮方針と成否・token・時間の関係を比較する．Liu[2] は状態の再取得費用を分け，予算をそろえた保持や外部からの状態復元を検証する．短縮が総費用削減を保証しないという一般論は既に指摘されている．

CodeAct[3] はコードを行動として用い，CaveAgent[4] は持続する実行状態を扱う．HD-Gist[5] は API 引数と値を分けた学習表現を生成中に参照する．Tool Search Tool[6] は初期表示を抑え，検索したツールの定義を展開する．段階的開示の研究 [7] も，同じ文書・チャンクで索引の配置と階層を比較する．状態保持，API 説明の分割，索引の保持は，いずれも既存の設計要素である．

Zhang ら [8] は科学計算について，値の生成，モデルへの提示，最終回答を分け，計算済みの同じ状態から返却方式を変える継続比較を行う．本稿が扱うのは，説明を一度取得した後の履歴表示と，操作先アプリの保存状態を変えない完了通知である．対象と介入は異なるが，段階を分けた診断や同一途中状態からの比較自体を独自方式とはしない．先行方式を同一条件で再実装した性能比較でもない．

3　状態と費用の定義

継続点の状態を会話履歴 H，Python 名前空間 P，操作先環境 E，残予算 B とする．表示への介入は H の一部に限り，P と E への操作権限をそろえる．終了手続きの診断では，作業対象アプリの状態と，完了を記録する supervisor の状態を区別する．保存ファイルの一致は観測範囲での一致であり，非公開の採点項目を推定するものではない．

仕事全体の論理トークンを

$$T_{\mathrm{work}} = T_{\mathrm{prefix}} + \sum_{j=1}^{n} (I_j + O_j) \tag{1}$$

とする．共通前半の実測値を条件ごとに一度加え，キャッシュ入力も I_j へ含める．継続だけの値も併記し，初回入力の削減量を全体の節約量へ転記しない．API，時間，実験用復元，評価，推論器のロード・常駐は別記する．評価器単独の秒数は表示比較では未計測で，後述の完了診断では計測した．表示比較における推論器稼働時間と継続・復元時間との差を，評価器だけの時間とはし

ない.

4　表示比較の方法

4.1　一覧と詳細を分けた表示比較

API 説明の省略による変化を調べるため，全表示 F，詳細のみ省略 D，同じ詳細と一覧を省略 B を設けた．古い成功出力のうち，保存した API 返却と表示処理から出自を照合できる区間だけを変更する．操作コード，リテラル，混合出力，エラー，最新応答を保持し，D と B で詳細の省略集合と省略印の形式をそろえる．Python 状態，公開スキーマ，原文の参照経路，公開契約の拒否時返信は共通である．従って，実行環境から知識を消去する介入ではない．

以前に観測済みの 4 課題の第 8 応答後を使い，各 3 条件を一度ずつ実行する計画を固定した．上限は各 16 生成，継続 120,000 token，480 秒とし，初回は入力キャッシュを再利用しない．全入力と残予算を計数して出力枠を制限する．独立評価器で保存環境の最終成否を判定し，仕事全体の論理トークンを $T_{\mathrm{prefix}} + \sum_j (I_j + O_j)$ とする．制御，計数，失敗，再取得を含め，復元・評価・推論器常駐は別記する．使用量不明の生成は実行も再試行もせず当該条件を終了する．API 作用と片付けが既知の場合のみ最終状態を評価するが，不明費用は補完しない．

推論には MiniMax-M3 の IQ4_XS 量子化版と llama.cpp b11669 を用いる．コンテキスト上限は 65,536，温度 1.0，top-p 0.95，出力上限 4,096 とした．継続の seed は 50 から呼出しごとに 1 増やし，条件間でそろえる．API 上限には前半の要求も含め，写真は 160，他の課題は 80 とする．順序は課題ごとに FDB，DBF，BFD，FBD で固定した．量子化版，推論器版，実装版，入力と返却のハッシュを保存し，実行条件を固定したことと結果の決定性を区別する．

主要な対照は D 対 F と D 対 B である．返金課題には追加で省ける一覧がなく，D/B の基礎入力が同じ負の対照となる．ここで生じた差は一覧保持の効果と解釈しない．既知 4 課題での開発比較のため，独立 12 課題や統計的な非劣性の証明とは扱わない．

4.2　対象と入力の照合

AppWorld[9] の写真整理，電気代精算，返金，曲一覧の書出を用いる．過去の失敗と入力計数を見た後の方式探索であり，独立の確認評価ではない．未閲覧の 12 シナリオは使わず，有利な課題への交換もしない．返却，保存 DB，Python 出力を照合するが，プロセスのヒープや乱数状態の完全複製はしていない．D/B で詳細の省略対象は課題順に 7・2・3・7 返却と一致し，B が追加で省く一覧は 1・1・0・3 返却だった．表 1 に示す初回入力は，共通の予算通知と省略印を含む．

表 1　通知を含む初回入力 token

課題	F	D	B
写真整理	8,789	7,342	6,848
電気代精算	11,267	11,009	10,202
返金	15,746	14,964	14,964
曲一覧の書出	13,771	12,142	9,473

4.3　採用基準と欠測

全 12 条件を評価し使用量が既知で監査に合格した場合に限り，D が F/B 双方の成功を失わず，合計 T_{work} が F より小さく B 以下なら，開発上の採用候補とする．一覧の追加価値を支持するには，入力の異なる課題で D 成功・B 失敗の対も必要とする．同数成功を統計的な非劣性とはみなさない．使用量不明はゼロや上限で埋めず，既知分と不明生成数を示す．不明がある対の正確な費用差は求めず，観測された既知分だけで判断できる範囲を記す．API 作用，復元，実行，片付け，評価，推論器版の不一致は全体を停止する．結果を見て seed・条件・予算を変更せず，未実行を成功や失敗に補完しない．

5　費用と成否の全結果

全 12 予定条件のうち 12 件を評価し，0 件が未評価，0 件が未実行だった．継続費用は既知 1,000,847 token，使用量不明は 1 生成だった．詳細省略 D は固定の採用基準を満たさず，品質を保った総費用削減として採用しない．D は既知分だけで F を 43,083 token 上回るため，未知分を除いても削減にはならない．表 2 に全予定条件を示す．D は写真で成功した一方，全 4 課題の既知費用が増えたため，この比較は品質を保った総費用削減を支持しない．また，未達で早く止まった条件を効率のよい成功とは数えない．

表 2　全 12 予定条件の成否と費用

課題	条件・成否	仕事全体 token	継続生成
写真整理	F・未達	145,850	8
	D・成功	120,935	6
	B・未達	165,015	11
電気代精算	F・未達	163,918	9
	D・未達	161,225	9
	B・未達	154,720	9
返金	F・成功	85,697	2
	D・成功	117,223	4
	B・成功	117,305	4
曲一覧の書出	F・未達	117,100	3
	D・未達	156,265+?	7
	B・未達	126,734	5

+? は別に使用量不明の 1 生成があることを示す.

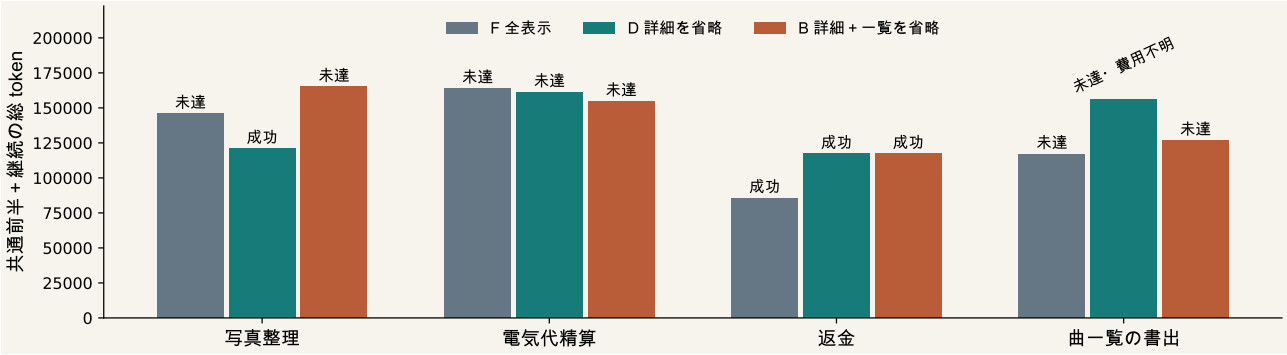


図 1　全 4 課題・3 表示の仕事全体 token と公式成否．共通前半を含み，縦軸は 0 から示す．斜線の棒は既知分のみで，さらに使用量不明の呼出しがある．

5.1　入力短縮と仕事全体の費用

全表示 F に対する全 4 課題の確定費用差は算出しない．一覧も省く B に対する全 4 課題の確定費用差は算出しない．表 3 は生成，API，継続時間の合計である．D の初回入力は F より全 4 課題で短いが，継続生成は 22 回から 26 回へ増えた．返金では全表示と詳細省略がともに成功した一方，初回入力 15,746 ／ 14,964 token に対し，継続費用は 32,209 ／ 63,735 token だった．この 1 回ずつの実行では，初回短縮と終了までの費用が逆方向だった．後続の行動も異なるため，各段階の独立した因果効果の和には分解しない．

表 3　全 4 課題合計の費用と実行量

指標	F	D	B
成功／評価済み	1/4	2/4	1/4
仕事全体・既知 token	512,565	555,648	563,774
継続・既知 token	302,185	345,268	353,394
使用量不明生成	0	1	0
前半＋継続生成	54	58	61
継続生成	22	26	29
前半＋継続 API	224	213	173
継続 API	173	162	122
復元 API	51	51	51
継続時間（秒）	1,308.5	1,354.0	1,359.1

継続 API は実際に送信した要求数であり，拒否され送信しなかった要求や原文参照は別に記録する．API 回数の減少だけで不要取得の削減とは判断せず，完了した処理と合わせて読む．本比較の推論器稼働全体は 4,112.859 秒で，復元・評価等の実験費用も生じた．

6　成否差と終了手続きの診断

写真整理の F/D は各 42 件の移動要求を送り，非 supervisor アプリの全保存ファイルが一致した．しかし公式成否は未達／成功で，完了通知の要求は 0 ／ 1 件だった．

完了通知だけを変えた追加診断．上の保存状態照合後，F/B の 2 状態を新しい模擬環境で再現し，完了 API のなし／1 回追加を固定 4 条件で比較した．送信済みの公開 API の返却と介入前 13 保存ファイルを全て照合し，既存 D と同じ空引数の通知だけを加えた．表 4 のとおり F は未達から成功へ変わり，その最終保存ファイルは元 D と全て一致した．B は通知後も未達だった．非 supervisor の保存ファイルは全条件で介入前後不変だった．

表 4　別環境の再現における終了手続きの介入

元状態	完了 API 追加	公式成否
F	なし	未達
F	1 回追加	成功
B	なし	未達
B	1 回追加	未達

再現 344 と介入 2 の計 346 API，独立評価 4 回，生成 0 で，全体 23.927 秒（評価 8.541 秒を含む）だった．これは既知 1 課題の事後診断であり，元の P55 成否・未知費用は変更しない．自動で完了を宣言してよい条件や，新たな自律エージェントの成功率は検証していない．この診断で，写真 F/D の成否差を写真整理そのものの改善と解釈する根拠はなくなる．一方，B は通知だけでは成功せず，完了宣言を一律に足せばよいともいえない．自動で通知を送るには，要求した仕事が済んだと判断する別の公開条件が必要である．

7　解釈の限界

既知 4 課題，単一モデル，各条件 1 回の開発比較であり，追加診断は同じ写真課題の事後介入である．診断 4 条件を独立 4 課題として数えない．seed と初回キャッシュ条件をそろえても，後続の実行経路の再現性は保証されない．返金 D/B の基礎入力は同じだが，後続の残時間通知などが異なるため，差を一覧の効果に帰属させない．過去 P47 と今回 P55 の全表示条件について，両方を評価できた比較のうち 1 課題で成否が異なった．こ

れは事前に反復数を割り付けた追試ではなく，変更の原因や分散の推定には用いない．

一覧を省いた B でも API 名は過去のコード等に残る．写真では一覧 26 名中 8 名，電気代では 26 名中 10 名が文字列として入力中に存在した．これは意味理解を測るものではなく，表示省略と情報の完全な喪失を区別するための照合である．省略対象も出自の確かな古い説明だけで，混合出力，最新応答，不確かな表示は残した．任意の履歴やモデルに同じ費用・成否の関係を保証するものではない．

8　結論

API 説明の履歴省略を，全予定条件の仕事全体の費用と終了手続きから調べた．詳細省略は初回入力を短くしたが，全 4 課題の既知費用だけでも全表示を上回った．また，写真整理では作業状態を保った完了通知により成否が変わった．この局所比較では，短い入力や成功数の増加だけを費用・実作業品質の改善根拠にはできない．必要情報と品質を守る制御を評価する際には，終了までの費用と，作業状態・終了手続きへの介入を対応させる必要がある．

参考文献

[1] R. Satish et al., “Beyond Token Savings: A Systematic Study of Context Compression in LLM Agents,” arXiv:2609.32961v1, 2026.

[2] S. Liu, “What Does Context Compression Cost an Agent? Interaction Costs Unrevealed by Task-Completion Metrics,” arXiv:2608.16370v1, 2026.

[3] X. Wang et al., “Executable Code Actions Elicit Better LLM Agents,” ICML, PMLR 235, pp.50208–50232, 2024.

[4] M. Ran et al., “CaveAgent: Transforming LLMs into Stateful Runtime Operators,” arXiv:2601.01569v4, 2026.

[5] Y. Jiang, M. Del Vecchio, M. Bansal, and A. Johannsen, “Hierarchical and Dynamic Prompt Compression for Efficient Zero-shot API Usage,” Findings of EACL, pp.2162–2174, 2024.

[6] Anthropic, “Introducing advanced tool use on the Claude Developer Platform,” Nov. 24, 2025.

[7] Y. He, Y. Zhao, J. Wang, and H. Chen, “Is Progressive Disclosure All You Need for Long-Context Agents?,” arXiv:2607.17598v1, 2026.

[8] K. Zhang et al., “Same Simulation, Different Evidence: Where Retrieved Tool Output Costs LLM Agents Accuracy,” arXiv:2610.26836v1, 2026.

[9] H. Trivedi et al., “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents,” ACL, pp.16022–16076, 2024.